Journal of Ambient Intelligence and Humanized Computing
https://doi.org/10.1007/s12652-021-03229-2

ORIGINAL RESEARCH

An adaptive honeypot using Q-Learning with severity analyzer

Shraddha Suratkar[1] · Kunjal Shah[2] · Aditya Sood[2] · Anay Loya[2] · Dhaval Bisure[2] · Umesh Patil[2] · Faruk Kazi[1]

Received: 13 June 2020 / Accepted: 25 March 2021
© The Author(s), under exclusive licence to Springer-Verlag GmbH Germany, part of Springer Nature 2021

Abstract
In this paper, a honeypot system has been presented, which conducts a severity analysis of the adversaries who attack it. The Honeypot systems are deployed by various organizations to protect their real systems from external threats. They consist of fake file-systems that remain aloof from the attackers. Honeypots gather logs of the attacks to protect the genuine systems from attackers. However, attackers also deploy honeypot detection tools. To defer detection from the attackers, a Q-learning based on an SSH-based honeypot named Cowrie has been implemented to make it adaptive and obtain as much information as possible about the intruder. Severity analysis has been implemented to classify attacks based on their severity. This can be used by real systems to enhance their firewalls, Intrusion Detection Systems, and other security mechanisms against these threats.

Keywords Honeypot · Q-Learning · Cowrie

1 Introduction

1.1 Honeypot

The definition of honeypot has been given as "a security resource whose value lies in being probed, attacked, or compromised". (Spitzner 2003). Honeypots are primarily used to provide warning and to retard, decelerate or stop automated attacks launched from external sources, and to record new exploits and vulnerabilities and gather information on new extra-organizational threats/dangers (Stolfo et al. 2008). Honeypot systems can be Internet services or completely functional computers that are exposed over the Internet so that they can be compromised and exploited. A Honeypot logs all the information related to the transitions with the attackers in a given session. Honeypots can be used to prevent distributed denial of service attacks too (Wang et al. 2019). Honeypots are dynamically evolving into adaptive interactive systems to match the development of the types of threats.

Honeypots have been classified into three different categories by Paliwal (2017), Mokube and Adams (2007) and Yeh and Yang (2008).

- *Low interaction*: In these honeypots, the interaction between the adversary and the decoy system is minimum because it is a static environment. These honeypots will generally emulate a small number of Internet protocols and network services, just enough to deceive the intruder. There is no real operating system involved here (Rahmatullah et al. 2016), and these types of honeypots are quite easy to design. Open ports like SQL, FTP, and HTTP are

✉ Shraddha Suratkar
sssuratkar@ce.vjti.ac.in

Kunjal Shah
kunjal1999@gmail.com

Aditya Sood
adityasood1998@gmail.com

Anay Loya
anayloya@gmail.com

Dhaval Bisure
dhavalbisure98@gmail.com

Umesh Patil
umeshhere007@gmail.com

Faruk Kazi
fskazi@el.vjti.ac.in

[1] Department of Electrical Engineering, Veermata Jijabai Technological Institute, Matunga, Mumbai 400019, India

[2] Department of Computer Engineering and Information Technology, Veermata Jijabai Technological Institute, Matunga, Mumbai 400019, India

Published online: 08 April 2021

emulated by these types of honeypots. Some examples include honeyd, honeybot, conpot.

- *Medium interaction*: These honeypots do not possess the capabilities of a real operating system. However, the services provided by them are technically more advanced when compared to the low interaction honeypot, as claimed by Ayeni et al. (2013). Since the medium interaction honeypots offer reduced interaction, they have lower chances of being compromised (Fraunholz et al. 2017). Medium interaction honeypots are very widely used today; Napenthes, Dioneae, andhoneytrap, and Cowrie, to name a few.
- *High interaction*: They are the most difficult as well as complicated to implement, deploy, and maintain. These honeypots give the intruder a real operating system environment which has no constrictions. (Nicomette et al. 2011; Catuogno et al. 2015). The networks on which the high-interaction honeypots are used should be isolated completely, and security precautions should be taken. If not isolated, the intruder can seize the system. As the services are run in a real operating system, it is harder for the intruder to detect the honeypot. Sebek and Argos are some instances of high-interaction honeypots.

Honeyword is a special kind of honeypot. Honeywords are decoy passwords associated with each user account, and they contribute a promising approach to detecting password leakage (Wang et al. 2018; Juels and Rivest 2013).

1.2 Cowrie

Cowrie is a honeypot that lies between the SSH and TELNET band. It is specifically developed for recording brute force attacks and the shell interaction performed by the adversary. As specified, it operates in two modes: medium interaction (shell) and high interaction (proxy). It emulates a UNIX system in Python in the former, whereas in the latter, it behaves like an SSH and telnet proxy to observe adversary behavior.

Kippo is developed to record brute force attacks and the entire shell interaction conducted by the adversary. Kippo provides a fake file system similar to Debian Linux capable of adding as well as removing files. Primitive tools such as:

Cat is added to misguide the adversary. Also, session logs stored in a UML compatible format could be replayed with the original prompt timings. Cowrie has carried on with Kippo development, which has expanded the software using SFTP/SCP support. The main inspiration for Kippo was Kojoney. All of the discussed honeypots have been developed in Python programming language as well as make use of bash scripts.

Cowrie logs are maintained in the following format:

{“eventid”, “macCS”, “timestamp”, “session”, “keyAlgs”, “message”, “system”, “isError”, “srcip”, “version”, “dstip”, “compCS”, “sensor”, “encCS”}.

2 Literature survey

Reinforcement Learning (RL) throws light on a huge space of learning problems characteristic of self-dependent agents making various interactions in an environment: sequential decision-making problems with a delayed reward. Reinforcement learning algorithms have been described by Lavrac and Dzeroski (1994) and they mainly involve the learning of a policy (which connects states and actions), that achieves the maximum reward over time. Reinforcement learning has been used at multiple places from medical image segmentation (Tian et al. 2020) to mobile fog computing (Jazayeri et al. 2020) and from vehicles, (Wang et al. 2020) to underwater acoustic sensor networks (Hemavathy and Indumathi 2020).

Researchers have shown great interest in the fields of reinforcement learning and honeypots. Dowling et al. (2018) has given ideas on using the concept of reinforcement learning for concealing honeypots. The starting bot commands shall try returning known honeypot responses or positives to fake requests. The bot shall then cease functionality or change actions, based upon the response tracked by it. However, Dowling’s research is having a very narrow focus because there is no interaction showing evident human cognition like late responses, typing flaws, etc.

However, there exist multiple honeypots set up to detect malicious items using already known vulnerabilities at multiple application as well as protocol levels (Bringer et al. 2012). Holz and Raynal (2005) shows tools that check honeypots set up on various attack vectors.

Wagener et al. (2011) proposes a high-interaction honeypot that can learn from adversaries and dynamically amend its actions using reinforcement learning. This honeypot can tactically block programs from running, entice the attacker by replacing programs with pseudo ones and affront adversaries to get knowledge of his personal attitude and race (Shi et al. 2016).

The very first self-adaptive honeypot system called adaptive honeypot alternative (AHA) was conceived by Wagener al. in 2011. Artificial Intelligence and the game-theoretic model played a crucial role in setting up honeypot behavior. The definition of honeypot payoff is the number of new unknown dangerous objects, in particular, the number of malicious files, as uploaded by the attacker. AHA was a Linux based operating system having an amended kernel module which permitted executing some particular actions inside an SSH server console. Here the honeypot selects actions with the probability defined by a predictor that has a very good payoff. The dataset of the set of previous decisions of the same honeypot is fed into this predictor

for training. Shi et al. (2016) presents an interesting concept of a fake honeypot system.

Here the defense system is composed of three components:

1. Honeypots
2. True services
3. Pseudo honeypot

The defense tries to make out the difference between the perpetrator and a genuine user. This proposed model was realized as a vulnerable FTP-server and was validated by simulation.

Pauna (2012) ameliorated AHA by adding rootkit malware detection capabilities. Two existent honeypot systems: one using Game Theory techniques to entice the attackers, the other based on detecting the malware downloaded and installed by the perpetrator were evaluated and a new approach was made by taking a combination of the two.

Pauna and Patriciu (2014) then designed a honeypot capable of learning from some perpetrators. The designed case adaptive SSH honeypot is based on Kippo and implements both Case Based Reasoning as well as Belief-Desire- Intention agents. Case-based reasoning is the technique of finding out a solution to new problems based on the decisions made while solving similar problem in history. The Belief-Desire-Intention agent model has a view of the world (Beliefs), several goals (Desires) and possible actions (Intentions). The actions are planned based on the experience that is gathered.

Pauna and Bica (2014) presented RASSH (Reinforced adaptive SSH honey- pot). This is a system consisting of a honeypot written in Python language and emulating the Secure Shell Server. The concept deals with the perpetrators and utilizes various reinforcement learning algorithms. In the authors' opinion, traps can dynamically change their responses to the attacker using reinforcement learning. This job shows the relationship with the idea of strategic decision-making using game-theoretic approaches. The honeypot system was using State-Action-Reward-State-Action algorithm for fixing on an action to be taken. However, RASSH is not a fully functional operating system yet.

Later QRASSH (Deep Q Learning Reinforced Honeypot) was proposed by Pauna et al. (2018) that uses deep Q-learning (DQN) algorithm for complete automation of the decision-making process. The environment had 57 states (Linux commands) and 5 action states. It used a simple ANN which had 64 input units, 2 hidden layers, and 5 output units each unit describing an action. It proposed 2 reward functions namely "simple" and "complex". The complex reward function had more subsets of the rewards being given. The result was that the performance of honeypot depends highly on the reward function. Also, the complex reward function took more time to train.

Lately, there has been the development of Interactive Adaptive Honeypots. The major function of Adaptive Honeypots is to be concealed from honeypot detection tools used malware deployers. This is achieved by using Reinforcement learning. The main purpose of the adaptive honeypot is increasing attack command transitions between the perpetrator and honeypot. This also favors a larger volume of data-set to be collected by honeypot which can be used for further analysis. The comparison of a normal honeypot and an adaptive honeypot was done. It was found that adaptive honeypot was able to increase the average transition between attacker and honeypot by 4 times (Dowling et al. 2018).

The major contributions of this paper are:

- Increasing attack command transitions between an attacker and honeypot using reinforcement learning.
- Concealing honeypots from honeypot detection tools deployed by the attacker.
- Determining the severity of each attack on Honeypot and storing them in a database.
- Mitigating an attack using captured patterns by honeypot.

The rest of the paper is outlined as follows: Sect. 3 describes the proposed methodology. Section 4 describes the prototypic implementation. The results of the models are explained in Sect. 5. Section 6 concluded the paper and Sect. 7 outlines future directions for research in this domain.

3 Proposed methodology

Honeypots are security resources that help attract, detect, and gather attack information. By pretending to be normal servers to attract attackers, honeypots can consume attackers' resources and time (Wang et al. 2017). The proposed solution uses the currently existent Cowrie, which is a medium interaction honeypot. In the cowrie source code, an Actions Module has been added to allow interaction with the attacker in the console. This adaptive honeypot has been deployed on an Amazon Web Services EC-2 instance. It uses Q Learning to increase the interaction with the attacker to avoid the detection of honeypot. This also increases the dataset collected by Honeypot.

Q learning is a very basic form of reinforcement learning wherein the behavior of the learning agent is altered based on the Q-values (action values) (Jang et al. 2019). The Q table is a data structure that maintains the action- state values. Before learning begins, Q value of Q-table is initialized to a possibly arbitrary fixed value. Then, for all time t when attackers inputs a command the honeypot selects a particular action p_t, receives a reward k_t, enters a new state z_{t+1} (which is dependent on both the previous state z_t as well as the selected action), and Q is updated.

$$\underbrace{NewQ(z,p)}_{New\ Q-value} = \underbrace{Q(z,p)}_{Current\ Q-value} + \underbrace{\propto}_{Learning\ Rate} \left[\underbrace{K(z,p)}_{Reward} + \underbrace{\gamma}_{Discount\ Rate} \overbrace{maxQ'(z',p') - Q(z,p)}^{\text{Maximum predicted reward, given new state and all possible actions}}\right]$$

After the interaction between a honeypot and attacker is terminated, the session, events, protocol are logged and passed to the severity analyser. The severity analyser gathers insights of the attack to prepare and store IOC (Indicators of Compromise) in the database. This will later be used by real-time systems in accordance with their needs.

The Architecture for the Interactive Honeypot is as shown in Fig. 1. The architecture contains the following modules:

1. Attacker
2. Adaptive Honeypot
3. Data Preprocessor
4. Severity Analysis
5. Results

3.1 Attacker

The attacker is a potentially malicious and risky person who attempts unauthorized access to the system. The attacker (human/bot) tries to access the existing system with malicious intent. As any access to the honeypot is restricted, any request can be considered to be an attack. The attacker interacts with the honeypot via commands. Attackers usually aim to compromise company and government workstations/servers. (Stolfo et al. 2008).

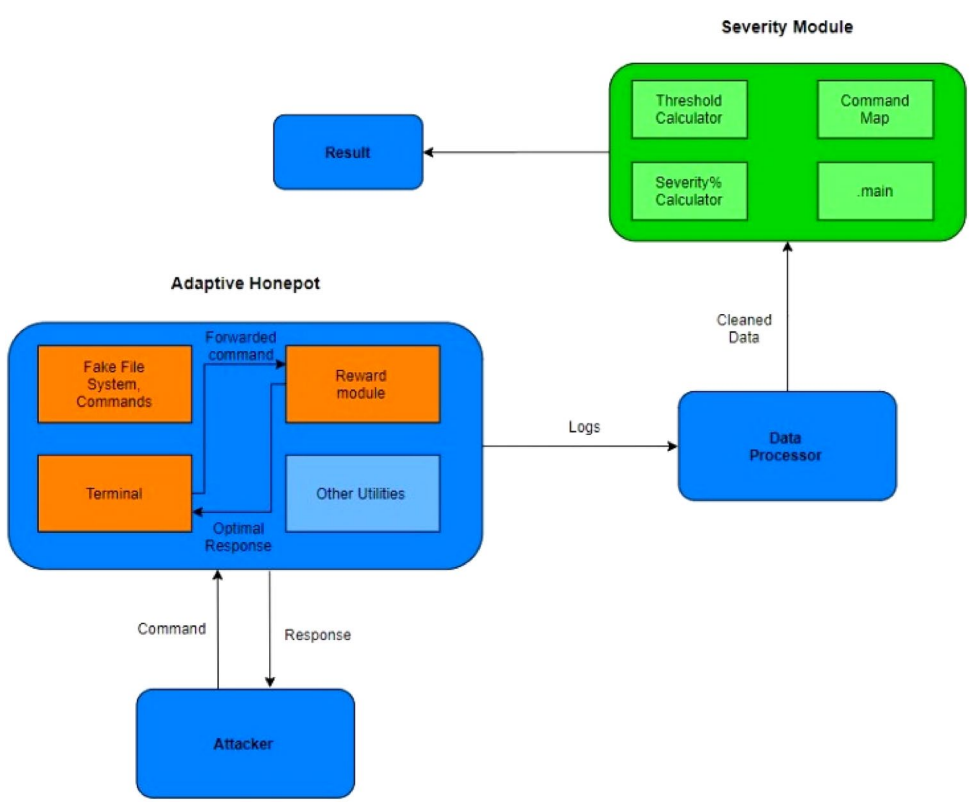

Fig. 1 The Proposed Architecture of Adaptive Honeypot

3.2 Adaptive honeypot

It depicts the Cowrie honeypot and modified its source code. The various algorithms for reinforcement learning have been used to increase the interaction with an attack sequence. The honeypot is rewarded for increasing interaction with the attacker which is evaluated using the final number of commands in the attack sequence. Here reinforcement learning is also used to find the optimum response to each command to avoid the attacker from terminating the session. Initially, actions taken by honeypot will be random (epsilon-greedy) and as more attacks happen, it will learn optimal values to fill the Q-table and take actions accordingly.

Attacker commands are categorized as:

- CheckConf :
  commands used to check configuration of system or its users
  (whoami, ps, uptime, ifconfig, uname, history, id)
- Install :
  command used to install new software
  (tar, unzip, mv, rm, cp,chmod, mkdir)
- Download :
  command used to download remote files
  (wget, ftp, curl, lwp-download)
- Run:
  command used to run a program
  (. /)
- Password:
  command used to change the password of the user logged
  (passwd)
- ChangeConf
  command used to change the state of the system
  (useradd, kill, nano, vim, sshd, userdel)

The actions to be implemented are a combination of the ones developed in the QRASSH system and the RASSH system and they contain:

1. Allow: Permits the command
2. Block: Stops the command
3. Delays: Delays the command
4. Substitute: Changes the output of a command or replaces the command
5. Insult: Changes the command output by displaying insult message

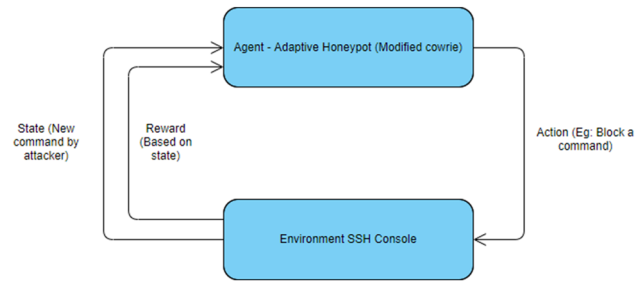

Fig. 2 Reinforcement learning model

The honeypot has several internal components:

1. Fake filesystems, commands: Honeypots are programs; hence a module is required where the logic for any command is implemented. A fake file system is also provided. After every session, the filesystem is recreated, so any files downloaded are deleted.
2. Terminal: A console, where input and output are displayed.
3. Reward Module: Any command received on the terminal is sent to the reward module, where the desired calculations are done, and an optimum action is provided. This also maintains a Q-table which keeps the reward for each (state, action) pair.
4. Other Utilities: They include the other files necessary for the working of honeypot. It also includes a database of the logs of each session.

3.2.1 Reward module

Q Learning has been implemented so that the honeypot uses adaptive technology. The reward module is a physical manifestation of the concept of reinforcement learning. The agent-environment model is shown in Fig. 2. The Q-Learning algorithm for the honeypot can be briefly described as follows:

1. Initialize $Q(z, p)$ to some constant value for all states z and actions p
2. Repeat for every case :
   (a) Initialize z as per the attacker's input
   (b) repeat for each step until z is terminal :
      i. Choose a action p for state z using policy function π (epsilon greedy)
      ii. Execute action p from above-described actions
      iii. Get the reward k using reward function and next state z_t
      iv. Update $Q(z, p)$ as using Q learning formula as described above
      v. Update state : $z = z_t$

The reward function for the reinforcement learning model is :

$$kt(z_i, p) = 1.0 \text{ if i is a subset of X}$$
$$= 0.5 \text{ if i is a subset of Y}$$
$$= 0.0 \text{ if i is a subset of Z or otherwise}$$

where $X = Run \cup Password$
$Y = Install \cup Download \cup ChangeConf$
$Z = CheckConf$

The intuition behind the actions can be described as follows. Attackers frequently download tools from attacked machines for illegal reasons. For instance, an attacker wants to download an illegal file and does so for abc.tar.gz. Once the download is complete he/she unzips it and runs the file using the command

```
./x
```

If the honeypot replaces this command with another known tool like SSH brute-force, it might make the perpetrator think that he/she downloaded the incorrect tool and he/she might also attempt to download something else. Due to this action, the honeypot discloses another malicious repository as well as an additional tool. In this way, more information about the attacker's resources can be achieved.

Insult messages are also included in the terminal as an action since it will confuse an attacker. An attacker might think that he/she was insulted by another attacker or the administrator. It can act as a reverse Turing test to check if the attack is automated.

3.3 Data preprocessor

The logs generated by the honeypot are stored in a MySQL database. The database initially consisted of two tables input and session as shown in Tables 1 and 2.

These tables are joined together on sessionID per IP address, to produce a new table as represented in Table 3. The join can be represented as:

$$R1 \rightarrow \pi_{<\text{session_id, ip, group_concat(input)}>}(input, sessions)$$

$$R2 \rightarrow \text{input} \infty_{<\text{input.sessions=sessions.id}>}(sessions)$$

The resultant result is given by $R \rightarrow R1(R2)$ (Bhanuprakash et al. 2014) Hence we get a list of commands executed per session of an ID address.

The severity module uses this table for efficient and simpler processing of data.

3.4 Severity analysis

Severity analysis is the process of understanding, processing and analyzing the data generated by the honeypot in order to detect patterns and gain useful knowledge. This is a new module that we have created to calculate the severity of each attack that has occurred on the honeypot. It returns the severity percent of a particular IP address.

Not much work has been done for severity analysis so far. Naik et al. (2018) has classified attacks based on severity as Imminent, Serious, Higher, Moderate and Lower.

The severity module in the figure has the following components:

Table 1 input

id	session	timestamp	realm	success	input

Table 2 sessions

id	startime	endtime	sensor	ip	termsize	client

Table 3 output

Session_id	ip	Input_sequence

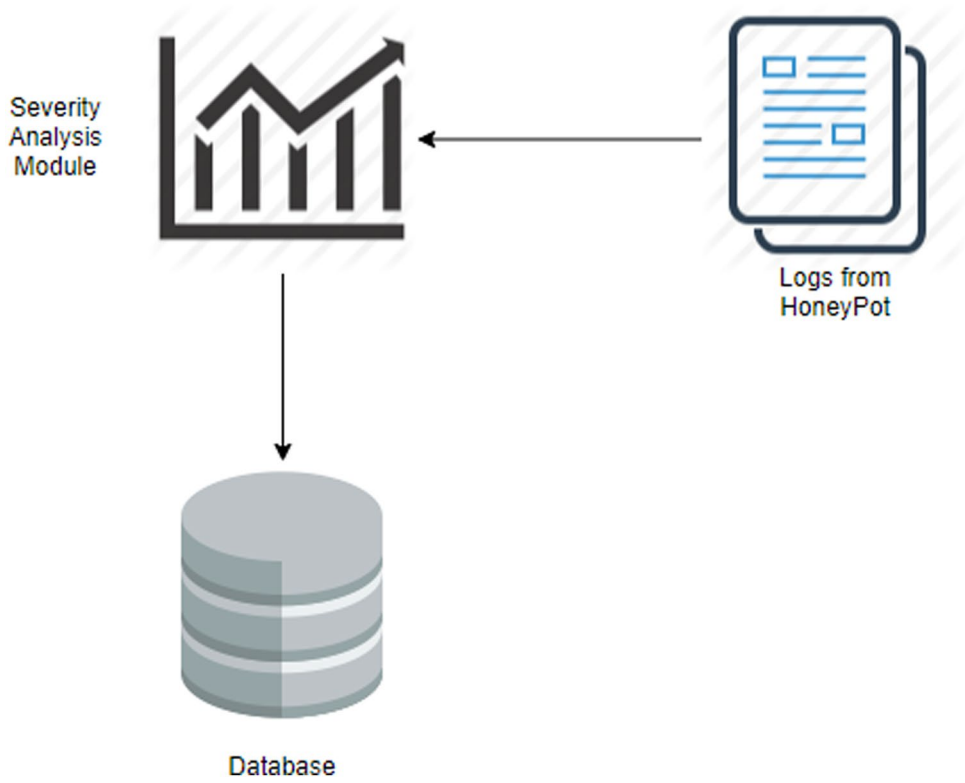

Fig. 3 Overview of Severity Analysis

1. Command Map: This consists of the commands present in the honeypot. Each command falls under a particular command group.
2. Threshold Calculator: This module finds the severity value by which attacks will be classified as severe or non-severe. This calculation is done by finding the least number of non-severe commands occurring before a severe command is executed successfully. The threshold is considered 2 times this number. This enables any attack containing a severe command to be flagged as severe as well too many non-severe commands to be flagged as severe.
3. Severity calculator: This module helps in finding the severity value of each command group. This is done before calculating the threshold value.
4. .main: This module is the main executing program which calls other modules as required. This returns the severity percent of each IP address, i.e. (severe attacks/ total attacks × 100)

3.4.1 Intuition behind severity

The Honeypot is assumed to be in State 'A' before an attack or perpetration. A 'State of a system' can be defined as the contents of a file system, configuration parameters, processes running, likewise. After an attack has taken place, the honeypot state is changed. The new state will have different contents of the file system, configuration parameters and processes running. The honeypot is now said to be in State 'B'.

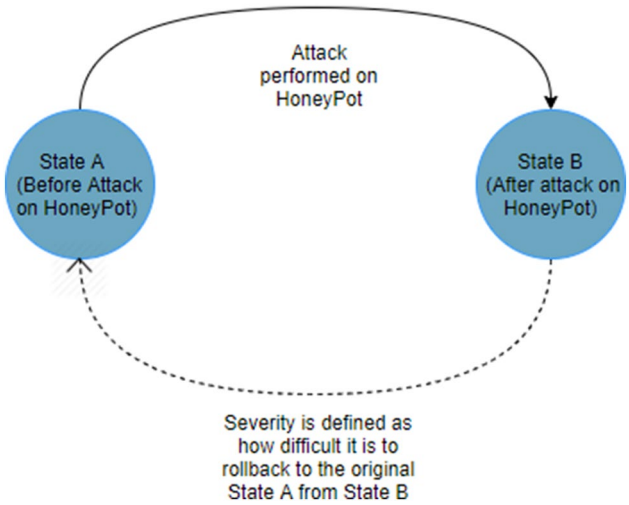

Fig. 4 Intuition behind severity of attacks

The severity of an attack is the measure of how difficult it is for a system to recover back to its original state. The commands which change the state of the honeypot permanently are assumed to be more severe as compared to the ones which can easily be rollbacked or do not end up affecting the system. For example, if a file is being downloaded onto the system, it

can be said that the change is permanent, hence it is a severe attack. Whereas an 'ls' command will display the contents of a particular directory, this does not affect the configuration of the system, hence it is termed as a non-severe attack. The overview of the severity analysis is given in Fig. 3.

3.4.2 Workflow of severity analysis

1. Logs from the MySQL Database of the Honeypot System are processed before being used by the severity analysis module as explained in Sect. 3.4.
2. The input to the severity analysis module was a MySQL table of the schema as shown in Table 3.
3. The commands are grouped in each attack according to the type of command as stated in the earlier sections.
4. Severity of each attack is calculated i.e. the sum of the severity for each command according to its group.
5. All attacks are classified based on their severity value. Classification is done by comparing the severity with an optimum threshold value. The threshold value is calculated using a method that will be described ahead.
6. Finally, the severity percent for each IP address is calculated. The formula for calculating the same can be given by:

$$Severity\ Percent = \frac{Number\ of\ Severe\ Attacks}{Total\ Attacks}$$

For example, there are two attackers with IP addresses 192.00.00.01 and 192.00.00.02. The first attacker performs a total of 100 attacks, out of which 50 attacks are severe. The second attacker performs a total of 200 attacks, out of which 25 attacks are severe. Hence the severity percent for the first attacker with IP address 192.00.00.01 is 50 / 100, which is 50%. Whereas the severity percent for the second attacker with IP address 192.00.00.02 is 25 / 200, which is 12.5%.

The output is stored in the form of a tuple (IP Address, SeverityPercent, MostSeverePattern) in a MySQL database instance, connected to the honeypot. Figure 4 shows the intuition behind the severity of attacks.

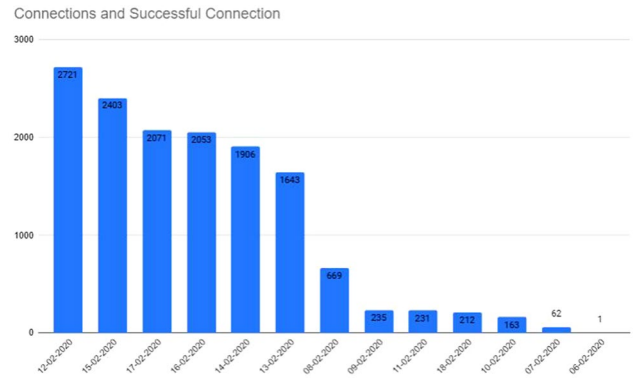

Fig. 6 Increasing probes per day

3.4.3 Optimum threshold value

1. Severity Values are assigned to each group of commands in the following order

CheckConifg < *ChangeConfig* < *Download* < *Install* < *Password* < *Run*

2. The minimum number of commands needed to reach a 'severe' command is found for all the attacks. CheckConfig is the only non-severe group of commands.

   Threshold value = least commands before 'Severe command' * 2.
3. Hence, we find an optimal threshold value for a 'severe command' which is the sum of minimum commands preceding a severe command in a given set of attacks. We do so because this ensures that any command which is higher than or equal to 'ChangeConfig' in severity gets treated as a severe attack whereas more number of non-severe commands also results in a severe attack.

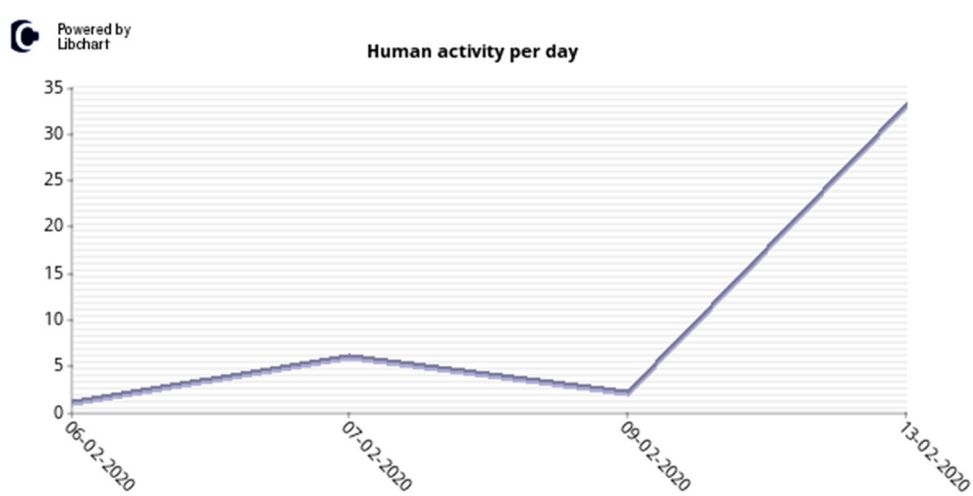

Fig. 5 Increasing attacks per day

Table 4 Attacks stored per IP

IP address	Geolocation	Sessions count	Severe source	Last Seen
49.88.112.114	Nanjing, China	831	Yes	2020–02-12 07:28:23
49.88.112.110	Nanjing, China	669	Yes	2020–02-22 18:37:28
49.88.112.118	Nanjing, China	297	Yes	2020–02-08 17:30:07
141.98.81.150	Panama,Central America	271	Yes	2020–02-22 18:54:43
141.98.81.138	Panama,Central America	211	Yes	2020–02-18 14:14:22
3.135.9.190	Fairfield, United States	92	No	2020–02-09 17:55:43
222.186.175.217	Nanjing, China	61	No	2020–02-22 18:54:43

Table 5 Sessions stored for each IP

Timestamp	IP	Sessions	User	Password	Success
2020–02-12 07:28:25	49.88.112.114	40633b244d6911ea…	root	1234	Yes
2020–02-12 05:02:18	49.88.112.114	d5cd22e84d5411ea…	root	01,234	No
2020–02-12 07:28:06	49.88.112.114	33ccb9da4d6911ea…	root	123@!@#	No
2020–02-12 06:31:45	49.88.112.114	5619f1404d6111ea…	root	passwd1234	No
2020–02-12 06:13:45	49.88.112.114	d235cab84d5e11ea…	root	google123	No
2020–02-12 05:47:39	49.88.112.114	2be657704d5b11ea…	root	admin1234	Yes

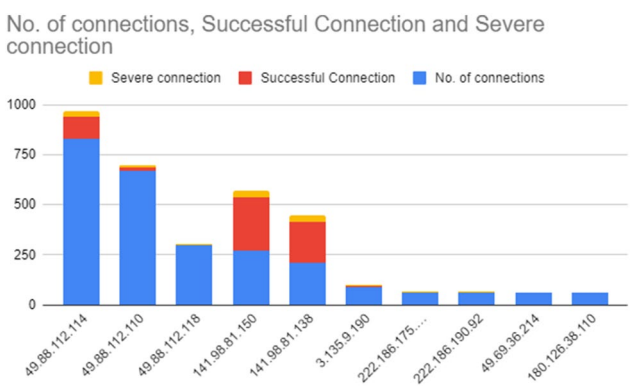

Fig. 7 Number of severe connections

Table 6 Comparison of cumulative attack transition

No. of attacks	This honeypot	Dowling's honeypot
10	60	100
20	110	200
30	260	480
40	580	700
50	1320	1100
60	1800	1420
70	2100	1900
80	2330	2250
90	2850	2700

3.4.4 Scope of severity analysis

The results of the severity analysis module in the project gives the severity per cent of each IP address. This information can be utilized by firewalls to improve a real system's security. Firewalls are used to filter incoming and outgoing traffic, adhering to a given set of rules. The predefined rules can be changed in order to prevent any severe attacks on the system. For example, if an IP address has a severity per cent of 75% it would be advisable to block such an IP address. Intrusion Detection Systems(IDS) could also use the results of the severity analysis module. IDS can analyze packets, if a notorious IP address is encountered then the event can be logged. This event can be analyzed for traffic signatures and patterns to learn about attackers who may use different IP addresses later.

4 Implementation

A prototype of the proposed architecture was developed and deployed for testing.

4.1 Preparing the honeypot

The base source code for the Cowrie honeypot was obtained and modified to support interactions with the attacker. The honeypot features a full fake file system resembling a Debian 5.0 installation for imitating a real system and a sub-routine to provide responses decided by the reward modules. The instance allows to put fake contents in the system and also

Table 7 Comparison of approximate number of commands to converge towards optimal Q values

Command	Action	This honeypot	Dowling's honeypot
wget	allow	25	65
wget	block	22	65
wget	substitute	28	65
sudo	allow	6	10
sudo	block	35	60
sudo	substitute	55	68

stores all the data downloaded/uploaded from the attacker for later inspection.

4.2 Deploying

The above-mentioned prototype was deployed on a virtual machine hosted on an Amazon EC2 Instance. The access points are configured to allow ssh and telnet connections to the server (honeypot) from the Internet. Access points are then made public to allow interactions and begin the collection of data.

5 Results

The proposed adaptive honeypot was deployed on an AmazonWebService-EC2 instance and it was live for 12 days from 6th February 2020 to 18th February 2020. Using kippo-graph for visualization the following results were observed- The number of attacks increased per day going from 5 attacks on 6th February to around 33 attacks on 13th February (Fig. 5). There was an increase in the number of probes per day on the Honeypot with the maximum number being in the second week (Fig. 6). The attacks performed on honeypot were logged and stored in the database in a format as shown in Table 4. For each row in Table 4, a corresponding table as shown in Table 5 was stored to log all the sessions per IP address.

Figure 5 depicts the number of unique connections per IP address, only a number of attacks were able to successfully login. Out of those successful logins some were classified as severe attacks by the proposed severity analysis module. The ten most connections from a unique address are shown in Fig. 7.

An important consideration is the security of the reward module. If it is compromised then various dangerous issues can occur (Pavate et al. n.d.) hence proper security measures are required to take care of the vulnerability to attacks (Shah et al. 2020, Kumar et al. 2020, Nerurkar et al. 2020, Kumar et al. 2020).

5.1 Comparison with high interaction honeypot

The number of cumulative attack transition recorded by the honeypot were more than other honeypot as proposed by Dowling et al. (2018) after around 40 attacks were crossed and more attacks were recorded as shown in Table 6. The no of attacks to converge towards optimal Q values, which give better interaction and learning policies, of various actions for common honeypot commands was less compared to the other high interaction and adaptive honeypots by Dowling et al. (2018). This is shown in Table 7. This means the honeypot learns faster about a optimal learning policy which gives more interaction resulting in more data. Also the severe connections recorded by the adaptive honeypot as defined by the severity module provide more information about the attacker than other connections recorded by high interaction honeypots.

5.2 Identity analysis of attacker

The attacks on the honeypot were performed by various locations around the world mostly from China and USA. Each attack is differentiated or categorized by a session ID which can be seen as a combination of IP address and a timestamp (Fig. 7). Attacks are also identified based on the response time to different actions to understand their behavior. Very quick response time or instant action indicates automated bot attack whereas a little slower response indicates a human attacker especially during actions such as "Insult". To make honeypot more valuable, we must attract and identify more human attackers than bots which provide far less information. Hence a human attacker can be uniquely identified by his session ID, type of response, and time taken for the response to actions taken by the proposed honeypot. More interactive behavior of attacker i.e. different types of responses e.g. an attacker trying other methods to attack after a 'block' action and other actions is the main aim of an adaptive honeypot so that it uses this behavior analysis of attackers to study about latest attacks.

5.3 Honeypot forensics

An attacker after compromising the adaptive honeypot is said to leave "fingerprints" behind which can be used for identification. In the proposed model, these fingerprints are the system activities i.e., how the state of the system was changed or what tools have been used by an attacker to perform malicious activities. As in severity analysis, the state change is used to determine the severity of the attack, similarly, a state change can be used to identify the attacker as the same type of attacks would lead to the same state. The state of the system is changed as commands are executed.

The different sequence of commands implies different attackers even though the severity could be the same. Hence common patterns are found out in the command sequence to identify different attackers. Patterns would consist of commands and tools used in a sequence. This is because most attacks follow a sequence of events that can be explored. Being an adaptive honeypot, it is possible that there are more than one possible frequent patterns from single IP as previous attack sequence was taking too long (due to delay action), getting blocked(due to block action) or not getting required results (due to substitute action). Hence an adaptive honeypot helps us get various techniques used by a single attacker (categorized by IP).

To trace an attacker, the unique tools used in the sequence can be identified and matched in any other attacks where they have been used. The final state of the system can be checked and compared with other attacks. The collected information can be analyzed further and compared with the information collected from other honeypots to help develop an identity of an attack and attribute it to an attacker.

6 Conclusion

As a closing word, honeypots are deployed to increase the security of real-time systems. This is achieved by data mining or analysis of the collected data-set by the honeypot. However, to counter the malware development, deployers have been using honeypot detection tools. Various methods have been proposed to improve adaptive honeypot, concealment techniques, etc.

An adaptive honeypot significantly increases its performance in working against attackers. An adaptive honeypot proposed in this paper can be used to increase attacker-honeypot transitions. Also, the dataset collected by an adaptive honeypot can then be used to carry out severity analysis of the attacks. This provides the real-time systems with a database consisting of some insight into the attacks that have taken place.

7 Future scope

In the RL module of the adaptive honeypot, most of the work has been done by hard-coding the reward function. There is scope of using reverse reinforcement learning to find the best reward function for increasing interaction with the attackers.

There is also scope for using other RL optimization methods like A2C, PPO which can give promising results when there is more number of attacks.

More work can be done on the honeypot forensics so that the backtracking process is made easier by utilizing the data collected in a better way.

References

Ayeni O, Alese B, Omotosho L (2013) Design and implementation of a medium interaction honeypot. Int J Comput Appl 975:8887

Bhanuprakash, C., Nijagunarya, Y. & Jayaram, M. (2014) A simple approach to sql joins in a relational algebraic notation. Int J Comput Appl 104(4)

Bringer ML, Chelmecki CA, Fujinoki H (2012) A survey: recent advances and future trends in honeypot research. Int J Comput Netw Inform Security 4(10):63

Catuogno, L., Castiglione, A. & Palmieri, F. (2015) A honeypot system with honeyword-driven fake interactive sessions. In: 2015 international conference on high performance computing & simulation (HPCS), IEEE, pp. 187–194

Dowling, S., Schukat, M. & Barrett, E. (2018) Using reinforcement learning to conceal honeypot functionality. In: Joint European conference on machine learning and knowledge discovery in databases, Springer, pp. 341–355

Fraunholz, D., Krohmer, D., Anton, S. D. & Schotten, H. D. (2017) Investigation of cybercrime conducted by abusing weak or default passwords with a medium interaction honeypot. In: 2017 international conference on cyber security and protection of digital services (Cyber Security), IEEE, pp. 1–7

Hemavathy, N. & Indumathi, P. (2020) Deep learning-based hybrid dynamic biased track (dl-hdbt) routing for under water acoustic sensor networks. J Ambient Intell Human Comput pp 1–15

Holz T, Raynal F (2005) Detecting honeypots and other suspicious environments. In: Proceedings from the sixth annual IEEE SMC information assurance workshop, IEEE, pp 29–36

Jang B, Kim M, Harerimana G, Kim JW (2019) Q-learning algorithms: A comprehensive classification and applications. IEEE Access 7:133653–133667

Jazayeri F, Shahidinejad A, Ghobaei-Arani M (2020) Autonomous computation offloading and auto-scaling the in the mobile fog computing: a deep reinforcement learning-based approach. Journal of Ambient Intelligence and Humanized Computing pp. 1–20

Juels A, Rivest RL (2013) Honeywords: making password-cracking detectable. In: Proceedings of the 2013 ACM SIGSAC conference on computer & communications security, pp 145–160

Kumar A, Abhishek K, Ghalib MR, Nerurkar P, Shah K, Chandane M, Bhirud S, Patel D, Busnel Y (2020) Towards cough sound analysis using the internet of things and deep learning for pulmonary disease prediction. Transactions on Emerging Telecommunications Technologies p e4184

Kumar A, Abhishek K, Shah K, Patel D, Jain Y, Chheda H, Nerurkar P (2020) Malware detection using machine learning. Iberoamerican Knowledge Graphs and Semantic Web Conference, Springer, pp. 61–71

Lavrac N, Dzeroski S (1994) Inductive logic programming. Springer, WLP, pp 146–160

Mokube, I. & Adams, M. (2007) Honeypots: concepts, approaches, and challenges. In: Proceedings of the 45th annual southeast regional conference, pp. 321–326

Naik N, Jenkins P, Savage N (2018) Threat-aware honeypot for discovering and predicting fingerprinting attacks using principal components analysis. In: 2018 IEEE symposium series on computational intelligence (SSCI), IEEE, pp 623–630

Nerurkar P, Busnel Y, Ludinard R, Shah K, Bhirud S, Patel D (2020) Detecting illicit entities in bitcoin using supervised learning of

ensemble decision trees. In: Proceedings of the 2020 10th international conference on information communication and management, pp 25–30

Nicomette V, Kaaniche M, Alata E, Herrb M (2011) Set-up and deployment of a high-interaction honeypot: experiment and lessons learned. J Comput Virol 7(2):143–157

Paliwal S (2017) Honeypot: A trap for attackers. Int J Adv Res Comput Commun Eng 6(3):842–845

Pauna A (2012) Improved self-adaptive honeypots capable of detecting rootkit malware. In: 9th international conference on communications (COMM), IEEE, pp. 281–284

Pauna A, Bica I (2014) Rassh-reinforced adaptive ssh honeypot. In: 10th international conference on communications (COMM), IEEE, pp. 1–6

Pauna A, Patriciu VV (2014) Casshh–case adaptive ssh honeypot. In: International conference on security in computer networks and distributed systems, Springer, pp. 322–333

Pauna A, Iacob A-C, Bica I (2018) Qrassh-a self-adaptive ssh honeypot driven by q-learning. In: International conference on communications (COMM), IEEE, pp 441–446

Pavate A, Kumawat D, Pansambal S, Nerurkar P, Bansode R (n.d.) Machine learning under attack: literature survey. Mach Learning 14, 14–18

Rahmatullah DK, Nasution SM, Azmi F (2016) Implementation of low interaction web server honeypot using cubieboard. In: International conference on control, electronics, renewable energy and communications (ICCEREC), IEEE, pp 127–131

Shah K, Bhandare D, Bhirud S (n.d.) Face recognition-based automated attendance system. In: International conference on innovative computing and communications, Springer, pp. 945–952

Shi L, Zhao J, Jiang L, Xing W, Gong J, Liu X (2016) Game theoretic simulation on the mimicry honeypot. Wuhan Univ J Nat Sci 21(1):69–74

Spitzner, L. (2003) Honeypots: tracking hackers. vol. 1, Reading: Addison- Wesley.

Stolfo S, Bellovin S, Hershkop S, Keromytis A, Sinclair S, Smith S (2008) Insider attack and cyber security. Advances in information security, ISBN 978-0-387-77322-3

Tian Z, Si X, Zheng Y, Chen Z, Li X (2020) Multi-step medical image segmentation based on reinforcement learning. J Ambient Intell Human Comput Online First 2020:1–12

Wagener G, State R, Engel T, Dulaunoy A (2011) Adaptive and self-configurable honeypots. In: 12th IFIP/IEEE international symposium on integrated network management (IM 2011) and workshops, IEEE, pp. 345– 352

Wang K, Du M, Maharjan S, Sun Y (2017) Strategic honeypot game model for distributed denial of service attacks in the smart grid. IEEE Trans Smart Grid 8(5):2474–2482

Wang D, Cheng H, Wang P, Yan J, Huang X (2018) A security analysis of honeywords. NDSS. https://doi.org/10.14722/ndss.2018.12345

Wang X, Guo N, Gao F, Feng J. (2019) Distributed denial of service attack defense simulation based on honeynet technology. Journal of Ambient Intelligence and Humanized Computing pp. 1–16

Wang, Z., Wan, Q., Qin, Y., Fan, S. & Xiao, Z. (2020) Research on intelligent algorithm for alerting vehicle impact based on multi-agent deep reinforcement learning. Journal of Ambient Intelligence and Humanized Computing pp. 1–11

Yeh C-H, Yang C-H (2008) Design and implementation of honeypot systems based on open-source software. In: IEEE international conference on intelligence and security informatics. IEEE, pp. 265–266

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.